# Purpose

# KIT-0 — kit/ Purpose + boundaries

## Piece A — Purpose

**File:** `Axion/libraries/kit/KIT-0_purpose.md`

---
library: kit
id: KIT-0
schema_version: 1.0.0
status: draft
---

# KIT-0 — kit/ Purpose + Boundaries

## Purpose
`kit/` defines the **kit packaging contract**: the deterministic structure of the output bundle produced in S9.
A kit is what an executor or operator consumes to build/run/deploy with minimal ambiguity.

## What it governs (in scope)
- Kit folder tree contract (what must exist, where it goes)
- Kit manifest schema (authoritative index of kit contents)
- Versioning rules (kit_version, schema versions)
- Export rules (internal vs external kit; restricted content stripping)
- Entry points (what file to start with; how to interpret the kit)

## What it does NOT govern (out of scope)
- Template content and render rules → `templates/`
- Proof ledger semantics → `verification/`
- Policy meaning and approvals → `policy/`
- Orchestration stage order/run manifest semantics → `orchestration/`

## Consumers
- Kit packaging stage (S9)
- External/internal executors (to run builds)
- Operator UI (download/export kit, inspect manifest)
- Gate evaluation (ensure kit includes required artifacts)

## Determinism requirements
- Kit contents are derived only from pinned artifacts produced during the run.
- Manifest is the single source of truth for kit contents.

- Kit tree is stable and versioned.

---

## Piece B — Boundary checklist

**File:** `Axion/libraries/kit/KIT-0_boundary_checklist.md`

---
library: kit
id: KIT-0a
schema_version: 1.0.0
status: draft
---

# KIT-0a — Boundary Checklist

Belongs in `kit/` if it answers:
- “What does the output bundle look like?”
- “What files must be included and how are they referenced?”
- “How do we export safely (internal vs external)?”

Does NOT belong in `kit/` if it answers:
- “How do we select templates?” (templates)
- “What are the proofs?” (verification)
- “How do we evaluate gates?” (gates)

Say **“next”** for **KIT-1 Kit folder tree contract**.

# Kit structure model

# KIT-1 — Kit folder tree contract (output structure)

## Piece A — Kit tree model (concept)

**File:** `Axion/libraries/kit/KIT-1_kit_tree_model.md`

---
library: kit
id: KIT-1
schema_version: 1.0.0
status: draft
---

# KIT-1 — Kit Folder Tree Model

## Purpose
Define a deterministic folder structure for packaged outputs so executors and operators always know where to look.

## Core idea
- The kit tree is stable across runs (only contents vary).
- Every file in the kit must be listed in KIT_MANIFEST.
- No “mystery files” and no directory scanning required.

## Minimum kit top-level structure
- kit_manifest.json (authoritative index)
- README.md (entry guidance)
- artifacts/ (pinned artifacts: canonical, standards, plans, proofs)
- docs/ (rendered templates)
- scripts/ (verification/run scripts, optional)
- metadata/ (run manifest snapshot, pins, environment notes)

---

## Piece B — Machine-readable kit tree spec

**File:** `Axion/libraries/kit/registries/kit_tree.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "kit_tree.v1",
  "folders": [
```

```
    { "path": "docs", "required": true, "purpose": "Rendered templates (human-facing docs/specs)." },
    { "path": "artifacts", "required": true, "purpose": "Core run artifacts (canonical, standards snapshot, plans, gate reports, proof ledger)." },
    { "path": "metadata", "required": true, "purpose": "Pinned refs, run manifest snapshot, and kit-level metadata." },
    { "path": "scripts", "required": false, "purpose": "Optional scripts for verification/build/run." }
  ],
  "files": [
    { "path": "kit_manifest.json", "required": true, "purpose": "Authoritative manifest of kit contents." },
    { "path": "README.md", "required": true, "purpose": "How to use this kit." }
  ]
}
```

---

## Piece C — Determinism rules (tree)

**File:** `Axion/libraries/kit/KIT-1_determinism_rules.md`

```
---
library: kit
id: KIT-1a
schema_version: 1.0.0
status: draft
---

# KIT-1a — Determinism Rules (Kit Tree)

- Folder and file list is defined only by kit_tree registry (no discovery).
- Paths are relative to kit root.
- A kit build must create required folders/files even if empty (except where forbidden).
- All included files must be represented in KIT_MANIFEST.
```

---

## Piece D — Validation checklist

**File:** `Axion/libraries/kit/KIT-1_validation_checklist.md`

```
---
library: kit
id: KIT-1b
```

schema_version: 1.0.0
status: draft
---

# KIT-1b — Validation Checklist

- [ ] kit_tree registry exists and is pinned
- [ ] required folders exist in output kit
- [ ] required files exist (README.md, kit_manifest.json)
- [ ] no files exist outside manifest (strict mode optional)

Say **“next”** for **KIT-2 Kit manifest schema**.

# Kit manifest schema

# KIT-2 — Kit manifest schema (authoritative index of contents)

## Piece A — Kit manifest model (concept)

**File:** `Axion/libraries/kit/KIT-2_kit_manifest_model.md`

```
---
library: kit
id: KIT-2
schema_version: 1.0.0
status: draft
---

# KIT-2 — Kit Manifest Model

## Purpose
Define the authoritative manifest that lists:
- what's included in the kit
- where each file lives
- what contract/artifact it corresponds to
- hashes (optional) for integrity
- export classification (internal/external safe)

## Design rules
- Manifest is the source of truth; do not rely on directory scanning.
- Manifest entries are deterministic and stable.
- Every included file has exactly one manifest entry.
```

---

## Piece B — Kit manifest schema (JSON Schema)

**File:** `Axion/libraries/kit/schemas/kit_manifest.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "kit_manifest.v1",
  "type": "object",
  "required": [
    "kit_id",
```

```
    "run_id",
    "kit_version",
    "created_at",
    "contents",
    "entrypoints"
  ],
  "properties": {
    "kit_id": { "type": "string", "pattern": "^KIT-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string" },

    "kit_version": { "type": "string", "minLength": 1 },
    "created_at": { "type": "string", "format": "date-time" },

    "export_class": { "type": "string", "enum": ["internal", "external"] },

    "entrypoints": {
      "type": "object",
      "required": ["readme", "primary_artifact"],
      "properties": {
        "readme": { "type": "string" },
        "primary_artifact": { "type": "string" }
      },
      "additionalProperties": false
    },

    "contents": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/item" }
    }
  },

  "$defs": {
    "item": {
      "type": "object",
      "required": ["path", "kind", "classification"],
      "properties": {
        "path": { "type": "string", "minLength": 2 },

        "kind": { "type": "string", "enum": ["doc", "artifact", "metadata", "script"] },

        "contract_id": { "type": "string", "description": "If this item corresponds to an artifact contract
ID" },
```

```
      "source": {
        "type": "object",
        "properties": {
          "produced_by_stage": { "type": "string" },
          "artifact_ref": { "type": "string" }
        },
        "additionalProperties": false
      },

      "hash": { "type": "string" },

      "classification": {
        "type": "string",
        "enum": ["public", "internal_only", "restricted"]
      }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

---

## Piece C — Determinism rules (manifest)

**File:** `Axion/libraries/kit/KIT-2_determinism_rules.md`

```
---
library: kit
id: KIT-2a
schema_version: 1.0.0
status: draft
---

# KIT-2a — Determinism Rules (Kit Manifest)

- contents entries are sorted deterministically:
  - kind order: metadata, artifact, doc, script
  - then path lexicographic
- Each path appears exactly once.
- contract_id values must match known artifact contract IDs when present.
- entrypoints are stable defaults:
```

- readme = README.md
- primary_artifact = artifacts/KIT_MANIFEST.json (or artifacts/CANONICAL_SPEC.json, pick one and keep stable)

---

## Piece D — Validation checklist

**File:** `Axion/libraries/kit/KIT-2_validation_checklist.md`

```
---
library: kit
id: KIT-2b
schema_version: 1.0.0
status: draft
---

# KIT-2b — Validation Checklist

- [ ] kit_manifest validates against schema
- [ ] contents list is non-empty and deterministically ordered
- [ ] no duplicate paths
- [ ] required entrypoints present and point to existing paths
- [ ] classification values set correctly
```

# Versioning + compatibility rules

You're right — I skipped it. Here it is as **KIT-3**, and I'll renumber the export section to **KIT-4** after this.

# KIT-3 — Versioning + compatibility rules

## Piece A — Versioning model (concept)

**File:** `Axion/libraries/kit/KIT-3_versioning_model.md`

id: KIT-3
library: kit
schema_version: 1.0.0
status: draft

# KIT-3 — Versioning + Compatibility Rules

## Purpose
Define how kit versions, schema versions, and compatibility guarantees work so:
- executors know what they are reading
- changes don't silently break consumers
- kits remain reproducible across time

## Version layers
1) **Kit format version** (kit_version)
- version of the overall kit structure and manifest contract

2) **Schema versions**
- kit_manifest schema version
- any embedded schema references (canonical_spec, standards_snapshot, etc.)

3) **Content versions**
- template versions
- standards pack versions
- pipeline version

## Compatibility guarantees (minimum)
- Same kit_version ⇒ same required folder tree + manifest fields
- Minor version bump may add optional fields/paths (backward compatible)
- Major version bump may change required structure/fields (breaking)

---

## Piece B — Compatibility matrix (rules)

**File:** `Axion/libraries/kit/KIT-3_compatibility_rules.md`

id: KIT-3a
library: kit
schema_version: 1.0.0
status: draft

# KIT-3a — Compatibility Rules

## Kit version semantics (recommended)
- MAJOR.MINOR.PATCH

### PATCH
- fixes typos/metadata
- no structural changes
- no required field changes

### MINOR
- may add new optional manifest fields
- may add new optional folders/files
- must not remove or rename required paths
- must not change meaning of existing fields

### MAJOR
- may add/remove/rename required paths
- may change required manifest fields
- requires executor update

## Schema version semantics
- Each JSON schema has its own schema_version.
- A kit can include multiple schema versions as long as the manifest references them explicitly and the executor supports them.

## Pinning requirement
- kit_manifest must include the kit_version
- run manifest snapshot must include pinned library/schema versions used to generate the kit

---

## Piece C — Machine-readable kit compatibility registry

**File:** `Axion/libraries/kit/registries/kit_compatibility.v1.json`

{

```
  "schema_version": "1.0.0",
  "registry_id": "kit_compatibility.v1",
  "kit_format": {
    "kit_version": "1.0.0",
    "requires": {
      "folders": ["docs", "artifacts", "metadata"],
      "files": ["kit_manifest.json", "README.md"]
    }
  },
  "schema_support": {
    "kit_manifest": ["1.0.0"]
  }
}
```

---

## Piece D — Determinism rules (version pinning)

**File:** `Axion/libraries/kit/KIT-3_determinism_rules.md`

id: KIT-3b
library: kit
schema_version: 1.0.0
status: draft

# KIT-3b — Determinism Rules (Versioning)

- kit_version is explicitly set and recorded in KIT_MANIFEST.
- kit_version changes require updating kit_compatibility registry.
- All referenced artifacts in the kit must be produced from pinned inputs recorded in the run manifest.
- Export class does not change kit_version; it changes manifest.export_class + filtered contents.

---

## Piece E — Validation checklist

**File:** `Axion/libraries/kit/KIT-3_validation_checklist.md`

id: KIT-3c
library: kit
schema_version: 1.0.0
status: draft

# KIT-3c — Validation Checklist (Versioning)

- [ ] kit_compatibility registry exists and is pinned
- [ ] KIT_MANIFEST includes kit_version
- [ ] required folders/files match kit_compatibility.requires
- [ ] schema_support lists supported kit_manifest schema versions
- [ ] version bumps follow MAJOR/MINOR/PATCH rules

If you want, I'll now continue with what I previously labeled **KIT-3 Export rules** as **KIT-4**, then **KIT-5 Kit gates**, then **KIT-6 Minimum viable kit set**.

# Export policies

# KIT-4 — Export rules (internal vs external, stripping restricted content)

## Piece A — Export policy model (concept)

**File:** `Axion/libraries/kit/KIT-3_export_rules.md`

```
---
library: kit
id: KIT-3
schema_version: 1.0.0
status: draft
---

# KIT-3 — Kit Export Rules (Internal vs External)

## Purpose
Define how kits are exported safely:
- internal kits may include internal-only knowledge/notes
- external kits must exclude restricted content and comply with use_policy rules

## Export classes
- internal: for internal executors/operators
- external: for third-party or untrusted execution environments

## What must be stripped for external export
- any item classified "restricted"
- any item classified "internal_only"
- any render envelopes containing restricted knowledge citations (if policy requires removal)
- any reuse excerpts beyond allowed limits (must not ship)

## What must remain
- public docs and artifacts required to execute the run (as allowed)
- a manifest that reflects what is included
- pinned provenance refs (but no secret material)
```

---

## Piece B — Export filter registry (machine-readable rules)

**File:** `Axion/libraries/kit/registries/kit_export_filter.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "kit_export_filter.v1",
  "rules": [
    {
      "rule_id": "EXPORT-EXTERNAL-STRIP-RESTRICTED",
      "export_class": "external",
      "deny_classifications": ["restricted", "internal_only"]
    },
    {
      "rule_id": "EXPORT-EXTERNAL-ALLOW-PUBLIC",
      "export_class": "external",
      "allow_classifications": ["public"]
    }
  ]
}
```

---

## Piece C — Determinism rules (export)

**File:** `Axion/libraries/kit/KIT-3_determinism_rules.md`

```
---
library: kit
id: KIT-3a
schema_version: 1.0.0
status: draft
---

# KIT-3a — Determinism Rules (Export)

- Export behavior is derived only from:
  - kit_manifest classifications
  - pinned kit_export_filter registry
  - pinned policy decisions (if policy blocks external export)
- External export produces a new kit_manifest with:
  - export_class = external
  - contents filtered deterministically
- Filter evaluation order is registry order; deny rules apply before allow rules.
```

---

## Piece D — Validation checklist

**File:** `Axion/libraries/kit/KIT-3_validation_checklist.md`

---
library: kit
id: KIT-3b
schema_version: 1.0.0
status: draft
---

# KIT-3b — Validation Checklist

- [ ] kit_export_filter registry exists and is pinned
- [ ] deny rules strip restricted/internal_only on external export
- [ ] external kit_manifest contains only allowed classifications
- [ ] external export behavior is deterministic and recorded
- [ ] policy can deny external export (KIT_EXPORT hook) and that decision is recorded

Say **“next”** for **KIT-4 Kit gates (tree valid, manifest valid, export compliant)**.

# Gates

# KIT-4 — Kit gates (tree valid, manifest valid, export compliant)

## Piece A — Kit gates intent

**File:** `Axion/libraries/kit/KIT-4_kit_gates.md`

---
library: kit
id: KIT-4
schema_version: 1.0.0
status: draft
---

# KIT-4 — Kit Gates

## Purpose
Ensure the packaged kit is structurally correct, fully indexed, and safe to export.

## Gate set (minimum)

### KIT-GATE-01 — Kit tree conforms
- kit_tree registry pinned
- required folders exist
- required files exist (README.md, kit_manifest.json)

### KIT-GATE-02 — Kit manifest valid
- KIT_MANIFEST exists
- validates against kit_manifest.v1
- contents list is deterministic and has no duplicate paths

### KIT-GATE-03 — Manifest matches filesystem (strict mode optional)
- every manifest entry path exists on disk
- no extra files outside manifest (if strict enabled)

### KIT-GATE-04 — Required artifacts included
At minimum, internal kit must include:
- artifacts/CANONICAL_SPEC
- artifacts/STANDARDS_SNAPSHOT
- artifacts/BUILD_PLAN
- artifacts/WORK_BREAKDOWN
- artifacts/ACCEPTANCE_MAP

- artifacts/GATE_REPORT
- artifacts/PROOF_LEDGER
- metadata/RUN_MANIFEST snapshot (recommended)

(Exact filenames can be contract-based; the gate checks by contract_id presence.)

### KIT-GATE-05 — Export classification correct
- internal kits may include internal_only items
- external kits must include only public items
- export filtering applied per kit_export_filter registry

### KIT-GATE-06 — Policy compliance for export
- if export_class == external:
  - KIT_EXPORT policy hook must allow export (or approved override recorded)

---

## Piece B — Mapping to pipeline stage + final checks

**File:** `Axion/libraries/kit/KIT-4_gate_mapping.md`

---
library: kit
id: KIT-4a
schema_version: 1.0.0
status: draft
---

# KIT-4a — Mapping

Primary enforcement occurs in:
- S9_KIT_PACKAGE (build kit + manifest)
- final verification prior to release/close (S10)

Kit gates should be evaluated:
- after S9 completes
- before any external export action

---

## Piece C — Machine-readable gate spec (optional)

**File:** `Axion/libraries/kit/KIT-4_kit_gates.spec.json`

{
  "schema_version": "1.0.0",
  "contract_id": "KIT-4b",
  "gates": [
    { "gate_id": "KIT-GATE-01", "name": "Kit tree conforms", "severity": "hard_stop", "checks": ["kit_tree pinned", "required folders/files exist"] },
    { "gate_id": "KIT-GATE-02", "name": "Kit manifest valid", "severity": "hard_stop", "checks": ["KIT_MANIFEST exists", "schema_valid(kit_manifest.v1)"] },
    { "gate_id": "KIT-GATE-03", "name": "Manifest matches filesystem", "severity": "warn", "checks": ["manifest paths exist", "no extra files if strict"] },
    { "gate_id": "KIT-GATE-04", "name": "Required artifacts included", "severity": "hard_stop", "checks": ["required contract_ids present in manifest"] },
    { "gate_id": "KIT-GATE-05", "name": "Export classification correct", "severity": "hard_stop", "checks": ["classifications comply with export_class + export filter"] },
    { "gate_id": "KIT-GATE-06", "name": "Policy compliance for export", "severity": "policy_controlled", "checks": ["if external export then policy allows or approved override"] }
  ]
}

---

## Piece D — Evidence requirements

**File:** `Axion/libraries/kit/KIT-4_evidence_requirements.md`

---
library: kit
id: KIT-4c
schema_version: 1.0.0
status: draft
---

# KIT-4c — Evidence Requirements (Kit Failures)

On failure, evidence must include:
- missing required folders/files (paths)
- manifest validation errors (field pointers)
- missing contract_ids (list)
- files on disk not present in manifest (if strict)
- policy decision refs for external export deny/approval
- remediation:
  - rebuild kit
  - fix manifest generation
  - fix classification tags

- re-run S9

---

## Piece E — Determinism rules

**File:** `Axion/libraries/kit/KIT-4_determinism_rules.md`

```
---
library: kit
id: KIT-4d
schema_version: 1.0.0
status: draft
---

# KIT-4d — Determinism Rules (Kit Gates)

- Gates run in fixed order KIT-GATE-01..06.
- Required artifacts are checked by contract_id, not filename guessing.
- Export filtering is deterministic and registry-driven.
- Evidence lists are deterministically ordered by path/contract_id.
```

---

## Piece F — Validation checklist

**File:** `Axion/libraries/kit/KIT-4_validation_checklist.md`

```
---
library: kit
id: KIT-4e
schema_version: 1.0.0
status: draft
---

# KIT-4e — Validation Checklist

- [ ] Kit gates cover tree, manifest validity, required artifacts, export classification, policy compliance
- [ ] Evidence pointers + remediation included
- [ ] Deterministic ordering applied to missing items lists
```

Say **“next”** for **KIT-5 Minimum viable kit set**.

# Minimum viable kit

# KIT-6 — Minimum viable kit set (runtime-required files + “done”)

## Piece A — MV kit inventory

**File:** `Axion/libraries/kit/KIT-6_minimum_viable_set.md`

---
library: kit
id: KIT-6
schema_version: 1.0.0
status: draft
---

# KIT-6 — Minimum Viable kit/ Set

## Goal
Define the smallest kit library set required to package deterministic outputs into a usable, auditable bundle.

## Required runtime files (minimum)
### 1) Core docs
- KIT-0_purpose.md
- KIT-0_boundary_checklist.md
- KIT-1_kit_tree_model.md
- KIT-1_determinism_rules.md
- KIT-1_validation_checklist.md
- KIT-2_kit_manifest_model.md
- KIT-2_determinism_rules.md
- KIT-2_validation_checklist.md
- KIT-3_versioning_model.md
- KIT-3_compatibility_rules.md
- KIT-3_determinism_rules.md
- KIT-3_validation_checklist.md
- KIT-4_export_rules.md
- KIT-4_determinism_rules.md
- KIT-4_validation_checklist.md
- KIT-5_kit_gates.md
- KIT-5_gate_mapping.md
- KIT-5_evidence_requirements.md
- KIT-5_determinism_rules.md
- KIT-5_validation_checklist.md

- KIT-6_minimum_viable_set.md

### 2) Schemas (must be in schema_registry)
- schemas/kit_manifest.v1.schema.json

### 3) Registries (runtime defaults)
- registries/kit_tree.v1.json
- registries/kit_compatibility.v1.json
- registries/kit_export_filter.v1.json

### 4) Gate specs (optional)
- KIT-5_kit_gates.spec.json

---

## Piece B — Definition of done (kit/)

**File:** `Axion/libraries/kit/KIT-6_definition_of_done.md`

---
library: kit
id: KIT-6a
schema_version: 1.0.0
status: draft
---

# KIT-6a — kit/ Definition of Done

kit/ is “done” when:

## Schemas + registries
- [ ] kit_manifest schema validates (JSON Schema check)
- [ ] kit_tree registry exists and is pinned
- [ ] kit_compatibility registry exists and is pinned
- [ ] kit_export_filter registry exists and is pinned

## Runtime behavior (contract-level)
- [ ] S9 produces a kit folder tree that matches kit_tree + kit_compatibility requirements
- [ ] kit_manifest.json is generated and validates
- [ ] Every included file is listed exactly once in kit_manifest.contents
- [ ] Required artifacts are present (by contract_id) and mapped to paths in the manifest
- [ ] Export filtering produces a deterministic external kit (when allowed)

## Safety

- [ ] External export strips internal_only + restricted content
- [ ] External export is blocked unless KIT_EXPORT policy hook allows (or approved override exists)

## Determinism
- [ ] manifest ordering is deterministic (kind order then path)
- [ ] version pinning is explicit (kit_version + schema support)
- [ ] kit contents are derived only from pinned run artifacts

---

### Piece C — Minimal folder tree

**File:** `Axion/libraries/kit/KIT-6_minimal_tree.txt`

```
Axion/libraries/kit/
  KIT-0_purpose.md
  KIT-0_boundary_checklist.md
  KIT-1_kit_tree_model.md
  KIT-1_determinism_rules.md
  KIT-1_validation_checklist.md
  KIT-2_kit_manifest_model.md
  KIT-2_determinism_rules.md
  KIT-2_validation_checklist.md
  KIT-3_versioning_model.md
  KIT-3_compatibility_rules.md
  KIT-3_determinism_rules.md
  KIT-3_validation_checklist.md
  KIT-4_export_rules.md
  KIT-4_determinism_rules.md
  KIT-4_validation_checklist.md
  KIT-5_kit_gates.md
  KIT-5_kit_gates.spec.json
  KIT-5_gate_mapping.md
  KIT-5_evidence_requirements.md
  KIT-5_determinism_rules.md
  KIT-5_validation_checklist.md
  KIT-6_minimum_viable_set.md
  KIT-6_definition_of_done.md
  KIT-6_minimal_tree.txt

  schemas/
    kit_manifest.v1.schema.json
```

```
registries/
  kit_tree.v1.json
  kit_compatibility.v1.json
  kit_export_filter.v1.json
```